DAY 25: MULTIPLE LINEAR REGRESSION

The Complete Mathematical Post-Mortem & Multi-Day Ledger Balancing Matrix

When factors (Features) become more than one, it is called Multiple Linear Regression. Let's completely dissect its inner math and store it perfectly in our minds!

Jab factors (Features) ek se zyada ho jaate hain, toh use kehte hain Multiple Linear Regression. Chaliye aaj iska poora post-mortem karte hain aur iske math ko dimaag mein ekdum sateek tarike se bhar dete hain!

---

1. THE REAL CHANGE: HOW THE EQUATION TRANSFORMED

Asali Badlav: Equation Ka Roop Kaise Badla?

Until now, our old shop was running on this simple formula:

Abhi tak hamari purani dukaan chal rahi thi is simple formula par:

$$Y = mX + c$$

Here, there was only one slope (m) and one factor (X). But when factors became more than one ($X_1, X_2, X_3...$), then every new factor will have its own personal slope (weight/coefficient)! Now the new look will look like this:

Yahan sirf ek dhalan (m) thha aur ek factor (X) thha. Lekin jab factors ek se zyada (X_1, X_2, X_3...) ho gaye, toh har naye factor ka apna ek personal dhalan (weight/slope) hoga! Ab naya roop aisa dikhega:

$$Y = m_1X_1 + m_2X_2 + m_3X_3 + ... + c$$

- Y: The output we need to find (Target).
  Y: Jo output hume dhoodhna hai (Target).
- X_1, X_2, X_3: Different independent factors (e.g., X_1 = Gold Weight, X_2 = Design Complexity, X_3 = Stone Count).
  X_1, X_2, X_3: Alag-alag factors (jaise X_1 = Gold Weight, X_2 = Design Complexity, X_3 = Stone Count).
- m_1, m_2, m_3: The personal slope or strength of each factor (Weights/Slopes). They tell how much a specific factor will shock the output.
  m_1, m_2, m_3: Har factor ki apni personal dhalan ya takat (Weights/Slopes). Yeh batate hain ki kaun sa factor output ko kitna jhatka dega.
- c: This is still the same old Intercept (Base value, when all factors become zero).
  c: Abhi bhi wahi purana Intercept hai (Base value, jab saare factors zero ho jayein).

---

2. GEOMETRIC TRANSFORMATION: LINE TO HYPERPLANE!

Geometry Ka Badlav: Line Se Seedha Hyperplane!

- **Simple Linear Regression**: A simple 2D line is formed on graph paper between X and Y.
  Simple Linear Regression: Graph paper par ek simple 2D Line banti thi (X aur Y ke beech).

- **Multiple Linear Regression**: When there are 2 factors (X_1, X_2) and one Y, the graph becomes 3D! And in 3D space, no line can be drawn; instead, a complete sheet or wall is formed, which in scientific terms is called a Hyperplane.
  Multiple Linear Regression: Jab 2 factors (X_1, X_2) aur ek Y hoga, toh graph banega 3D! Aur 3D space mein koi line nahi kheenchi ja sakti, wahan ek poori chadar ya deewar banti hai jise science ki bhasha mein Hyperplane kehte hain.

If factors become more than 3, we cannot see it with our eyes (4D, 5D space), but math keeps working perfectly behind the scenes.

Agar factors 3 se zyada ho gaye, toh hum use aankhon se dekh nahi sakte (4D, 5D space), par math parde ke piche bilkul sateek kaam karta rehta hai.

PART A: SINGLE DATA INSTANCE EVALUATION

The Mathematical Problem Setup (Single Row Math)

Let's assume we have a data point where we need to find the output Y (Gold Ornament Price) using two different inputs (X_1 and X_2):

Maan lijiye hamare paas ek data point hai jahan hume output Y (Gold Ornament Price) nikaalna hai do alag-alag inputs (X_1 aur X_2) ke jariye:

- **X_1 (Gold Weight)** = 10 Grams
  X_1 (Sone ka Weight) = 10 Grams

- **X_2 (Design Complexity Rating)** = Rating 3
  X_2 (Design Complexity Rating) = Rating 3

- **Actual Market Price (Y)** = 80,000 (The real price inside the data table)
  Actual Market Price (Asali keemat Jo data table me hai) = 80,000

Let's assume our computer has randomly guessed the initial starting parameters:

Maan lijiye hamare computer ne randomly shuruati parameters guess kiye hain:

- Initial Slope m_1 = 5000
- Initial Slope m_2 = 6000
- Initial Intercept c = 2000

- Learning Rate (α) = 0.0001

Let's see how the exact mathematics runs behind the scenes in this single iteration loop:

Chaliye dekhte hain ki exact mathematics is single iteration ke loop mein parde ke piche kaise chalti hai.

STEP 1: Computational Prediction (Y_pred)

First of all, the computer will apply our newly transformed Multiple Linear Regression equation:

Sabse pehle computer hamari naye roop wali Multiple Linear Regression equation lagayega:

$$Y\_pred = m_1X_1 + m_2X_2 + c$$

Let's put all the values inside it:

Isme saari values daal dete hain:

$$Y\_pred = (5000 * 10) + (6000 * 3) + 2000$$

$$Y\_pred = 50000 + 18000 + 2000$$

$$Y\_pred = 70,000$$

The computer guessed 70,000 whereas the actual price was 80,000.

Computer ne guess kiya 70,000 jabki asali keemat thi 80,000.

STEP 2: Instantaneous Loss Calculation (Error)

The rule for calculating the error is exactly the same old one:

Galti nikaalne ka niyam bilkul wahi purana hai:

$$\text{Error} = (\text{Actual Y - Predicted Y})$$

$$\text{Error Gap} = 80000 - 70000 = 10,000$$

$$\text{Squared Error Cost} = (10000)^2 = 100,00,00,000 \text{ (10 Crores)}$$

Asali keemat aur guess ke beech ka farq (gap) 10,000 aaya, jiska squared cost 10 Crores bana.

STEP 3: Calculus Gradients (The Slopes Direction)

Now comes the real fun! Because we have two different slopes (m), the gradient for both will be calculated separately by multiplying with their respective feature inputs (X):

Ab aayega asali maza! Kyunki hamare paas do alag-alag m hain, toh dono ki dhalan unke apne-apne X ke sath multiply hokar alag se nikalegi:

Gradient for m_1 (gradient_m1):

$$gradient\_m1 = -2 * X_1 * (Actual\ Y - Predicted\ Y)$$

$$gradient\_m1 = -2 * 10 * (80000 - 70000)$$

$$gradient\_m1 = -20 * 10000 = -2{,}00{,}000$$

Gradient for m_2 (gradient_m2):

$$gradient\_m2 = -2 * X_2 * (Actual\ Y - Predicted\ Y)$$

$$gradient\_m2 = -2 * 3 * (80000 - 70000)$$

$$gradient\_m2 = -6 * 10000 = -60{,}000$$

Gradient for Intercept c (gradient_c):

$$gradient\_c = -2 * (Actual\ Y - Predicted\ Y)$$

$$gradient\_c = -2 * (80000 - 70000) = -20{,}000$$

STEP 4: Parameters Simultaneous Update

Let's assume our Learning Rate (α) = 0.0001 (a small learning index so that it doesn't overshoot). Now all three parameters will change at the exact same moment:

Maan lijiye hamara Learning Rate (α) = 0.0001 hai (chota learning rate taaki overshoot na ho). Ab teeno parameters ek sath badlenge:

New m_1 Update:

$$m_1 = m_1 - (learning\_rate * gradient\_m1)$$

$$m_1 = 5000 - (0.0001 * -200000)$$

$$m_1 = 5000 + 20 \rightarrow m_1 = 5020$$

New m_2 Update:

$$m_2 = m_2 - (learning\_rate * gradient\_m2)$$

$$m_2 = 6000 - (0.0001 * -60000)$$

$$m_2 = 6000 + 6 \rightarrow m_2 = 6006$$

New c Update:

$$c = c - (learning\_rate * gradient\_c)$$

$$c = 2000 - (0.0001 * -20000)$$

$$c = 2000 + 2 \rightarrow c = 2002$$

Complete Mathematical Conclusion (Single Day)

Did you see, brother? After just a single iteration loop:

- m_1 increased from 5000 to 5020.
- m_2 increased from 6000 to 6006.
- c increased from 2000 to 2002.

Now in the next loop sequence, the computer will calculate the prediction using these new parameter coordinates ($m_1=5020, m_2=6006, c=2002$), which results in a prediction of **70,222**—which is much closer to our target of 80,000! The error is dropping steadily.

Aapne dekha bhai? Sirf ek single iteration ke baad: m_1 badhkar 5000 se 5020 ho gaya, m_2 badhkar 6000 se 6006 ho gaya, aur c badhkar 2000 se 2002 ho gaya. Ab jab agle loop mein computer in naye parameters ke sath prediction nikaalega, toh prediction 70,000 se badhkar 70,222 aayegi—jo ki hamare target 80,000 ke aur zyada paas hai! Galti dheere-dheere kam ho rahi hai.

PART B: MULTI-DAY BATCH DATA SOLUTION

1. Hamara Live Multi-Day Matrix Dataset

Now we are taking historical ledger records from two completely different days:

Hum do alag-alag dino ka ledger records le rahe hain:

- **Day 1 Record**: X_1 = 10 Grams, X_2 = 3 Rating → Actual Y = 80,000
  Day 1: X_1 = 10 Grams, X_2 = 3 Rating → Actual Y = 80,000
- **Day 2 Record**: X_1 = 30 Grams, X_2 = 4 Rating → Actual Y = 3,75,000
  Day 2: X_1 = 30 Grams, X_2 = 4 Rating → Actual Y = 3,75,000

Identical Initial State Configurations:

- Slope for weight (m_1) = 5000
- Slope for design (m_2) = 6000
- Intercept (c) = 2000
- Learning Rate (α) = 0.0001

2. STEP-BY-STEP BATCH PROCESSING (Parde Ke Piche Ka Math)

In batch multi-day processing, the computer calculates the individual prediction errors and calculus gradients for both days independently first.

Multi-Day reality mein computer pehle dono dino ka alag-alag prediction aur calculus dhalan nikaalta hai.

ROW 1: Day 1 Ke Data Ka Calculation

1. Prediction (Y_pred1):

$$Y\_pred1 = (m_1 * 10) + (m_2 * 3) + c$$

$$Y\_pred1 = (5000 * 10) + (6000 * 3) + 2000 = 50000 + 18000 + 2000 = 70,000$$

2. Gradients for Day 1:

$$grad1\_m1 = -2 * X_1 * (Actual\ Y - Y\_pred1) = -2 * 10 * (80000 - 70000) = -2,00,000$$

$$grad1\_m2 = -2 * X_2 * (Actual\ Y - Y\_pred1) = -2 * 3 * (80000 - 70000) = -60,000$$

$$grad1\_c = -2 * (Actual\ Y - Y\_pred1) = -2 * (80000 - 70000) = -20{,}000$$

ROW 2: Day 2 Ke Data Ka Calculation

1. Prediction (*Y_pred2*):

$$Y\_pred2 = (m_1 * 30) + (m_2 * 4) + c$$

$$Y\_pred2 = (5000 * 30) + (6000 * 4) + 2000 = 150000 + 24000 + 2000 = 1{,}76{,}000$$

2. Gradients for Day 2:

$$grad2\_m1 = -2 * X_1 * (Actual\ Y - Y\_pred2) = -2 * 30 * (375000 - 176000) = -60 * 199000 = -1{,}19{,}40{,}000$$

$$grad2\_m2 = -2 * X_2 * (Actual\ Y - Y\_pred2) = -2 * 4 * (375000 - 176000) = -8 * 199000 = -15{,}92{,}000$$

$$grad2\_c = -2 * (Actual\ Y - Y\_pred2) = -2 * (375000 - 176000) = -3{,}98{,}000$$

---

3. MULTI-DATA REALITY: Averaging and Balancing

According to the rules of batch learning, the computer will add the gradients of both days together and compute their Mean Average, so that a massive jump in a single record does not disrupt global stability:

Ab loop ke niyam ke mutabik, computer dono dino ki dhalan ko aapas mein jodkar uska Average (Mean) nikaalega taaki kisi ek din ke data ka jhatka pure model ko kharab na kare:

Average Gradient for m_1:

$$Avg\ Gradient\ m_1 = (grad1\_m1 + grad2\_m1) / 2$$

$$Avg\ Gradient\ m_1 = (-200000 + (-11940000)) / 2 = -12140000 / 2 = -60{,}70{,}000$$

Average Gradient for m_2:

$$Avg\ Gradient\ m_2 = (grad1\_m2 + grad2\_m2) / 2$$

$$\textbf{Avg Gradient } m_2 = (-60000 + (-1592000)) / 2 = -1652000 / 2 = -8,26,000$$

Average Gradient for c:

$$\text{Avg Gradient } c = (grad1\_c + grad2\_c) / 2$$

$$\text{Avg Gradient } c = (-20000 + (-398000)) / 2 = -418000 / 2 = -2,09,000$$

---

4. FINAL PARAMETERS SIMULTANEOUS UPDATE

In the final step, the algorithm multiplies these Mean Balanced Gradients by our safe Learning Rate ($\alpha = 0.0001$) to adjust all properties concurrently:

Ab aakhiri step mein, computer in Average Gradients ko hamare safe Learning Rate (α = 0.0001) se multiply karke teeno purani values ko ek jhatke mein update karega:

Naya m_1:

$$m_1 = 5000 - (0.0001 * -6070000) = 5000 + 607 \rightarrow 5607$$

Naya m_2:

$$m_2 = 6000 - (0.0001 * -826000) = 6000 + 82.6 \rightarrow 6082.6$$

Naya c:

$$c = 2000 - (0.0001 * -209000) = 2000 + 20.9 \rightarrow 2020.9$$

The Real Core Gist of These Numbers

In Numbers Ka Asali Nichod:

Did you see, brother? When multi-day records pass through, the computer beautifully balanced the weight equations across both environments:

- m_1 (Gold Weight multiplier) shot up directly from **5000 to 5607** because on Day 2, the price gap was huge, forcing the algorithm to aggressively scale up the weight priority.
- m_2 (Complexity factor) climbed steadily from **6000 to 6082.6**.
- c (Fixed baseline overhead cost) expanded from **2000 to 2020.9**.

Aapne dekha bhai? Do alag-alag dino ka data aane par computer ne dono ka math balance kiya: m_1 (Gold Weight) jhatke se 5000 se seedha 5607 pahunch gaya kyunki Day 2 par weight ka gap bohot bada thha, toh computer ne weight ki takat ko tezi se badhaya. m_2 (Complexity) dheere se 6000 se 6082.6 hua. c (Base cost) 2000 se badhkar 2020.9 ho gaya.